



INT Engineering Guidelines:

Specification-Driven Development/Delivery (SDD)

Audience: All Application Development engineers (Java, .NET, Node.js, Python, PHP, React, Angular, Flutter, React Native)

Status: v1.0

Date: 24-Jul-2026

A large, light blue circular watermark is centered on the page. Inside the circle, the letters "INT." are written in a large, white, bold, sans-serif font.



Table of Contents

1. Engineering Vision	4
2. AI-First Philosophy	4
3. Why Traditional SDLC Fails Here	5
4. What Is Specification-Driven Delivery (SDD)	6
5. INT Engineering Principles — Non-Negotiable	7
6. Engineering Lifecycle — The New SDLC	7
7. Discovery & Consulting	8
8. The Project Constitution — constitution.md Explained	10
9. Naming & Identifiers — Why the Slug Is Not Cosmetic	12
10. The Artefact Chain in Practice — BRD → Spec → Plan → Tasks	13
11. Specification Standards	16
11.1 Before writing	16
11.2 Authoring rules	16
11.3 On including API details and test cases in the spec	16
11.4 Template — .ai-context/specs/<feature-slug>.spec.md	17
12. Spec Peer Review — Gate 1	19
12.1 Who reviews	19
12.2 What "peer" means here	19
12.3 Turnaround and outcomes	20
12.4 Worked example	20
13. Architecture Standards (Plan Review, Gate 1 Continued)	21
14. Repository Standards	21
15. AI Workspace — .agent/ and .ai-context/ Explained	22
16. Prompt Engineering	24
17. Context Engineering	25
18. Development Standards	26
19. QA & Validation	26
19.1 One test_cases.md, or split it?	26
19.2 Standing rules	27
20. Security Standards	27
Worked example — 2fa-otp-login at Gate 2	28
21. State & Progress Management	29
21.1 Spec/task lifecycle states	29
21.2 .ai-context/status.md — the board	29

22. Documentation Standards	30
23. Release Management	31
24. Hotfix Process	32
25. Production Support	33
26. Engineering KPIs	35
27. Definition of Ready / Definition of Done	36
Definition of Ready (spec → development)	36
Definition of Done (implementation → merge)	36
28. Anti-Patterns	37
29. Templates	38
constitution.md — full skeleton	38
spec.md — full skeleton	39
plan.md — full skeleton	39
tasks.md — full skeleton	40
ADR — full skeleton	40
status.md — full skeleton	40
Hotfix spec — full skeleton	41
30. Checklists — Quick Reference	41
Gate 1 — Spec Peer Review	41
Gate 2 — Code Review	42
Security Checklist (used at Gate 2 and on every hotfix)	42
Release Checklist	42
Hotfix Checklist	43
Production Support / Incident Triage Checklist	43



1. Engineering Vision

INT builds software with AI coding agents as the default execution layer, not an occasional accelerant. The engineer's job shifts from typing every line to **specifying correctness precisely enough that generation is deterministic, and verifying the output against that specification** — not against a vague sense that it "looks right."

Three commitments follow from that, and everything else in this document is downstream of them:

1. The **specification is the primary engineering artefact** — reviewed, versioned, diffed like code. Code is generated output.
2. **No agent output ships without a named human accountable for it.** Ownership doesn't transfer to the tool.
3. **Speed comes from clarity, not from skipping steps.** A team that specs well ships faster than a team that types fast and fixes later.

2. AI-First Philosophy

"AI-first" does not mean "AI does the thinking." It means the engineering process is designed around the fact that generation is now near-free and the constraint has moved to intent and verification. In practice:

- **Agents are treated as literal-minded, context-dependent executors** — capable of excellent output when given precise, scoped instructions, and confidently wrong when given ambiguity. Assume the agent will do exactly what you wrote, not what you meant.
- **Context lives in the repository, not in a chat window.** Anything the agent needs to know twice belongs in `.ai-context/`, not retyped into every prompt.
- **Model choice is a tool decision, not an architecture decision.** Claude Code, Copilot, Gemini, Codex are interchangeable executors against the same spec/plan/tasks chain. If switching models changes what gets built, the spec was underspecified.

- The agent's speed is the reason discipline matters more, not less.** A slow typist who skips the spec produces one bad file. A fast agent who skips the spec produces a thousand lines of plausible-but-wrong architecture before anyone notices.

3. Why Traditional SDLC Fails Here

Traditional assumption	Why it breaks with AI agents
Design docs are written once, code is the expensive part	Code is now cheap to produce and expensive to <i>un-produce</i> if it's wrong — the design doc is now the expensive-to-get-wrong part
Requirements ambiguity gets caught in dev standups over days	An agent turns ambiguous requirements into a working-looking implementation in minutes, before anyone has a chance to catch the ambiguity
Code review is where quality gets enforced	By the time there's a diff to review, an agent has already made dozens of implicit design decisions no one signed off on — review is too late to be cheap
Documentation is written after the system is stable	A system built by prompting has no "after" — there's no design phase whose notes become the docs, unless something is deliberately captured before generation
Context lives in senior engineers' heads	An agent has none of that tacit knowledge unless it's written down; tribal knowledge that worked for onboarding a human doesn't transfer to a model session

The net effect of running AI-assisted delivery on an unmodified traditional SDLC: *vibe coding with better autocomplete* — fast to a demo, expensive to make production-real.



4. What Is Specification-Driven Delivery (SDD)

Definition: SDD is a delivery methodology where a structured, version-controlled specification — not the source code — is the primary artefact. Code, tests, and documentation are *generated from* the spec by a human-directed agent, and regenerated as the spec evolves, rather than the spec being reverse-engineered from code (or never written).

Four artefacts, each with a distinct owner and question:

Artefact	Question it answers	Owner
Constitution	What are our non-negotiables for every feature in this project?	Tech Lead / Architect — written once, amended rarely
Spec	What are we building, for whom, what does "correct" mean — including the API contract and its test cases?	Engineer + BA, reviewed by peers (Gate 1)
Plan	Given spec + constitution, what's the technical approach?	Engineer, reviewed at Gate 1
Tasks	What's the ordered, independently verifiable build sequence?	Generated from the plan, reviewed by engineer

What SDD is not: a longer prompt. A prompt dies when the chat window closes. A spec is a committed file that survives model switches, sprint boundaries, and engineer handovers.





5. INT Engineering Principles — Non-Negotiable

These sit in every project's [constitution.md](#). Projects may add stricter rules. None may weaken these.

1. **No implementation task runs without an approved spec.** Approval = a named peer reviewer who isn't the author (#12).
2. **Stop vibe coding.** No direct, spec-less prompting against a codebase on any branch that gets reviewed, merged, or demoed (#28).
3. **Test-first.** No implementation code before the corresponding tests exist, are reviewed, and are confirmed to fail (Red phase).
4. **The spec is the contract, not the code comments.** No AI-attribution in comments or commit messages — ever (traceability references to a spec/task ID are not attribution — see #16).
5. **Secrets and PII never enter a spec, plan, task, or prompt log.**
6. **Context is scoped, not dumped.** Feed the agent the spec, plan, and referenced modules for the current task — not the whole repo.
7. **A human is accountable for every merged line, regardless of who typed it.** "The agent wrote it" is not a defence in review, production, or postmortem.

6. Engineering Lifecycle — The New SDLC

None

Discovery/Consulting → Constitution (once) → Spec → **Gate 1 Peer Review** → Plan → Architecture check → Tasks → Test-first (Red) → Guided Implementation (Green) → **Gate 2 Review** → Merge → Release → Production Support → (feeds back into Spec/BRD)

Traditional phase	SDD equivalent	What changes
-------------------	----------------	--------------



Requirements gathering	Spec authoring	Written as testable acceptance criteria and an API contract, not interpretive prose
High/low-level design	Plan authoring	Captured <i>before</i> code exists, checked against constitution.md
Sprint/task breakdown	Task generation	Generated from the plan, reviewed by the engineer, not hand-written from memory
Development	Guided implementation	Engineer directs and reviews generation in small verifiable increments
Testing	Test-first generation	Tests generated from the spec's own test-case table <i>before</i> implementation tasks run
Documentation	Living spec	Spec/plan/status/prompt history are documentation, kept current by necessity
Code review	Two-gate review	Spec/plan peer review pre-code + code review post-generation
Status meetings	State & progress board	.ai-context/status.md (#21) replaces "what's everyone working on" as a standing question

7. Discovery & Consulting

Before a spec gets written, the engineer (with BA/PM where one exists) must close out discovery:

- **Confirm the business need**, not just the requested feature — flag it if they diverge, before scoping the spec.
- **Identify what's already decided vs. still open** — architecture constraints, integration contracts, design-system tokens, data ownership.



- **Capture the outcome in `.ai-context/BRD.md`** as a numbered entry — this is the source spec authoring pulls from; a spec should never be the first place a requirement is written down.
- **Get sign-off from whoever owns the decision** — architecture, security, product — before spec drafting starts.

Worked example — `.ai-context/BRD.md`:

markdown

None

BRD-014: Strengthen authentication for the policy holder portal

****Raised by:**** Compliance (regulatory audit finding, Q2 review)

****Business need:**** Reduce account-takeover risk for policy holders who can view and download personal and payment data through the portal.

****Sponsor:**** CISO; Product Owner, Policy Holder Portal

****Priority:**** High — audit remediation deadline 30 Sept

****Decided:**** Second factor is OTP via SMS, opt-in initially (existing Twilio integration, no new vendor).

****Open at BRD stage:**** Whether backup codes are needed for v1 — deferred to Security to decide before spec drafting.

****Notes:**** Mandatory (non-opt-in) rollout is a separate future BRD item, not in scope here.

Discovery is a gate, not a formality: if you can't answer "what problem, for whom, bounded how" in one paragraph, the spec is not ready to write.

8. The Project Constitution — `constitution.md`

Explained

The constitution is the one artefact that is *not* written per feature. It is written once per project (or once per major sub-system in a large polyglot estate), and it exists so



that every spec doesn't have to re-state rules that apply to all of them. Think of it as the difference between a law and a contract: the constitution is the law; each spec is a contract that operates inside it and never needs to restate "don't break the law" explicitly.

What belongs in it, and what doesn't: rules that are true for *every* feature this project will ever build, stated specifically enough that a Gate 1 reviewer can point to a line and say "the plan violates this." Vague aspirations ("write secure code") don't belong — they can't be checked. If a rule only applies to one feature, it belongs in that feature's spec, not here.

Worked example — [.ai-context/constitution.md](#) for the Insurance Platform:

markdown

None

Project Constitution — Enterprise Embedded Insurance Platform

Testing Discipline

- Test-first is mandatory for every API endpoint and every state-changing operation; no exceptions for "simple" endpoints.
- Minimum 80% line coverage for any service touching policy or payment data; 60% elsewhere. Coverage is a floor, not a target to write to.
- Node services: Jest. Any future Java services: JUnit 5 + Mockito.
- Frontend: React Testing Library, no snapshot-only tests for logic-bearing components.

Security Posture

- No PII (policy holder name, DOB, phone number, payment instrument, government ID) appears in logs at ANY log level, including debug.
- All customer-facing endpoints sit behind OAuth2 + the existing rate-limiter; no endpoint ships without a rate-limit decision made explicit in its plan (even if the decision is "none, and here's why").
- Secrets only via AWS Secrets Manager; never in `.env` files committed to any repo, sanitized or not.

Architectural Constraints

- Approved datastores: PostgreSQL (system of record), Redis (cache, session, rate-limiting only — never system of record). No new datastore without an ADR approved by the Architect.
- Approved messaging: Kafka. No direct service-to-service synchronous calls into the regulatory-reporting service from the request path — it is event-driven only, to protect request latency from its SLA.
- Frontend state management: Redux Toolkit only — no introducing a second state library "for this one feature."

Non-Functional Baselines

- p95 API latency < 400ms for customer-facing endpoints, measured at the gateway, not in application logs.
- 99.9% availability target for the policy issuance path.
- RPO 15 minutes / RTO 1 hour for PostgreSQL.

Versioning Rules

- Public APIs are semver. Breaking changes require a major version bump, an ADR documenting the break, and a minimum 90-day deprecation window communicated to distribution partners.

Why this matters more than it looks like it does: every Gate 1 review (#12) checks the plan against this file line by line. A plan that's silent on a constitution rule isn't neutral — it's a gap. If the constitution says "no PII in logs" and a plan doesn't mention logging at all, that's a question the reviewer asks, not an assumption they wave through.

Who owns it: the Tech Lead/Architect authors the first version at project kickoff (within 2 weeks, per the companion standard). Amendments go through the same review rigor as any spec — a constitution change is itself proposed as a short





spec-like change request, reviewed, and dated, because silently editing the law a project runs on is exactly the kind of drift this whole standard exists to prevent.

9. Naming & Identifiers — Why the Slug Is Not Cosmetic

Every feature gets a **slug** the moment its spec is created — a short, kebab-case, human-readable identifier (*2fa-otp-login*, not *login-fix* or *feature-42*). The slug is not a filename convenience; it is the **single thread that ties every artefact for that feature together**:

Where the slug appears	Example
Spec file	<i>.ai-context/specs/2fa-otp-login.spec.md</i>
Plan file	<i>.ai-context/plans/2fa-otp-login.plan.md</i>
Tasks file	<i>.ai-context/tasks/2fa-otp-login.tasks.md</i>
Test cases file	<i>.ai-context/test_cases/2fa-otp-login.test_cases.md</i>
Git branch	<i>feature/2fa-otp-login</i>
PR title	<i>[2fa-otp-login] Add OTP verification endpoint</i>
Status board entry	<i>.ai-context/status.md</i> , row key
ADR cross-reference	"supersedes decision made in <i>2fa-otp-login.plan.md</i> "
Release notes line	"OTP two-factor login (<i>2fa-otp-login</i>) — see spec for full AC"

A bad slug quietly breaks traceability months later. *login-fix* collides the moment a second login bug shows up; *feature-42* tells a reviewer, an agent, or a new hire



nothing and forces them to open the file to find out what it is. Rules for a good slug: 3–5 words, specific enough to be unique for the life of the project, verb-free (it names a thing, not an action), never reused even after a feature is deprecated (deprecated specs are archived, not deleted, and the slug retires with them).

Sub-identifiers, scoped under the slug, for everything inside the artefact chain:

- Acceptance criteria: `<slug>.AC1`, `<slug>.AC2`, ...
- API endpoints defined in the spec: `<slug>.API01`, `<slug>.API02`, ...
- Spec-derived unit test cases: `<slug>.UT01`, ...
- Tasks: `<slug>.T01`, `<slug>.T02`, ...
- ADRs are the one exception — they get project-global sequential IDs (`ADR-0007`) because they can outlive, and apply across, more than one slug.

This scheme is what makes #16's "prompt by identity" guidance possible — the ID is stable even when the surrounding prose is edited, so "implement `2fa-otp-login.T03`" always points at the same thing, but "implement the rate-limiting task" might not, once the tasks file has been revised twice.

10. The Artefact Chain in Practice — BRD → Spec → Plan → Tasks

This is the same feature (BRD-014, #7) carried through all three remaining artefacts, so the chain is concrete rather than described in the abstract.

Step 1 — Spec (full template in #11) is authored from BRD-014. Intent, acceptance criteria, and the API contract get written; nothing about *how* Twilio gets called or which service owns the rate-limit counter appears here — that's the plan's job.

Step 2 — Plan, derived from the spec, once Gate 1 has approved it:

markdown



None

Plan: OTP-Based Two-Factor Login

Derived From

.ai-context/specs/2fa-otp-login.spec.md

Architecture Approach

- New endpoints live in the existing `auth-service` (Node/Express) — no new service, per constitution's "no new datastore/service without ADR" spirit extended here by the reviewer's judgement call.
- OTP generation: `crypto.randomInt`, not a third-party package (see #20 for why).
- Twilio dispatch happens via the existing `notifications` Kafka topic the service already publishes to — async, not a direct HTTP call in the request path, so a slow Twilio response can't block login.

Data Model

- New Redis key: `otp:{otp\_session\_id}` → { attempt_count, expires_at }, TTL 15 minutes. No new PostgreSQL table — session state is transient and Redis is the approved store for exactly this per constitution.md.

Constitution Check

- [x] No new datastore introduced without ADR — none introduced.
- [x] Testing discipline matches constitution.md — Jest, test-first.
- [x] Security posture matches constitution.md — no PII in the Redis key (keyed by session id, not phone number); OTP never logged.

Explicitly Deferred

- Backup codes — BRD-014 marked this open; Security deferred it out of v1, so the plan deliberately does not build storage for it.

Sequencing

1. Redis session/rate-limit scaffolding
2. `/auth/otp/verify` endpoint + Kafka publish for dispatch



- 3. Rate-limit lockout logic
- 4. Frontend OTP entry screen

Step 3 — Tasks, generated from the plan's Sequencing section, each with a stable ID:

markdown

None

Tasks: OTP-Based Two-Factor Login

Derived From

.ai-context/plans/2fa-otp-login.plan.md

Sequence

- [] 2fa-otp-login.T01 — Add Redis session/rate-limit scaffolding
 - Acceptance: supports AC1, AC3
- [] 2fa-otp-login.T02 — Implement `/auth/otp/verify` + Kafka publish
 - Acceptance: 2fa-otp-login.AC2, 2fa-otp-login.API02
- [] 2fa-otp-login.T03 — Rate-limit lockout after 5 failed attempts
 - Acceptance: 2fa-otp-login.AC3
- [] 2fa-otp-login.T04 — Frontend OTP entry screen
 - Acceptance: 2fa-otp-login.AC1, 2fa-otp-login.AC4

Each task is independently generatable, independently reviewable, and independently mergeable — the engineer runs the agent against one task at a time (#16), not the whole tasks file in one prompt.

11. Specification Standards

11.1 Before writing



1. Can you describe the feature in one unambiguous paragraph a zero-context engineer could build from?
2. Have you gathered every artefact the agent needs — module it touches, prior related spec, API contract, design tokens, constraints carried forward?
3. Is this aligned with whoever needs to sign off?

If any answer is "not yet," don't open the agent.

11.2 Authoring rules

- **One feature, one spec.** "The user module" is a project. "OTP-based 2FA for existing users" is a spec.
- **Intent and acceptance criteria before any technical detail.** Testable conditions, not "improve security."
- **Link, don't restate context** — reference [.ai-context/](#) paths, don't paste unrelated blocks into the spec.
- **Name what's explicitly out of scope.**
- **If the feature exposes or consumes an API, the spec defines its contract** (#11.3) — the plan decides implementation, but payload shape, status codes, and exceptions are product decisions, not technical ones, and belong where product-level review happens.

11.3 On including API details and test cases in the spec

Verdict: yes, but conditionally. Folding the API contract (payload, exceptions) and the spec-derived test-case table into the spec — rather than leaving them to the plan or a separate QA document — is adopted as standard, for one reason: acceptance criteria that reference "the OTP endpoint" without a defined payload are not actually testable yet, they only look testable. Gate 1 reviewers were finding this gap at Gate 2 instead, which is exactly the expensive-review-shift-right pattern SDD exists to prevent.

The condition: this section is mandatory only when the feature has an API surface (an endpoint it exposes or a contract it consumes). A pure refactor or an internal



batch job has no API Contract section, and reviewers should not demand one where there's nothing to contract.

Where the line sits against `test_cases/` (#19): the spec's test-case table holds only the acceptance-level cases directly derived from its own AC — enough for QA and the agent to know the feature is correctly built. Broader QA concerns (negative testing sweeps, data-volume variations, device/browser matrices, cross-feature regression) still live in `.ai-context/test_cases/`, which references back to the spec rather than duplicating it.

11.4 Template — `.ai-context/specs/<feature-slug>.spec.md`

markdown

None

Spec: <Feature Name>

Spec ID

<feature-slug>

Status

<see #21 for the full state machine — Draft / In Peer Review / Approved / ...>

Linked BRD

.ai-context/BRD.md#BRD-NNN

Intent

<One paragraph: what changes, for whom, under what condition>

Context

- Builds on: .ai-context/architecture.md (<section>)
- Related: .ai-context/specs/<related-spec>.spec.md
- API contract (if consuming an external one): <path/link>

API Contract



<slug>.API01 — <METHOD> <path>****Request payload:****

```
```json
{ "field": "type" }
```
```

****Success response (<code>):****

```
```json
{ "field": "type" }
```
```

****Exceptions:******| Code | Condition | Response body |**

|---|---|---|

| 4xx | <condition> | <shape> |

Acceptance Criteria

1. <slug>.AC1 — Given <state>, when <action>, then <outcome>.
2. ...

Unit Test Cases (spec-derived)**| Test ID | Maps to AC | Scenario | Expected |**

|---|---|---|---|

| <slug>.UT01 | AC1 | <scenario> | <expected> |

Explicitly Out of Scope

- <item>

Non-Functional Constraints (from constitution.md)

- <latency / throughput / compliance constraint>

The full worked spec (2FA login, all sections populated) is in the companion standard, [INT_SDD_Standard.md #7.3](#), and threaded through #10 above.



12. Spec Peer Review — Gate 1

This is now the single most consequential review in the lifecycle — more so than code review — and the team should treat it that way, not as a formality on the way to "the real work." The reasoning: an approved spec deterministically generates code, tests, and documentation simultaneously. A miss here doesn't cost one bad diff; it costs a bad diff, bad tests that pass because they were derived from the same bad premise, and documentation that faithfully records the wrong intent. A code review miss is contained to that PR. A spec review miss propagates.

12.1 Who reviews

- Named reviewer, never the author.
- For features with an API Contract section, at least one reviewer should be someone who will also touch Gate 2 for the same feature, or a domain SME — someone reading for "does this contract make sense against what already exists," not just "is this readable."
- Security or Architecture sign-off is required, not optional, whenever the plan touches a constitution rule under Security Posture or Architectural Constraints.

12.2 What "peer" means here

Not one Tech Lead rubber-stamping — reading for genuinely different failure modes:

- **Ambiguity check:** could two competent engineers build materially different things from this intent?
- **Testability check:** is every acceptance criterion phrased as given/when/then, not an adjective ("secure," "fast," "robust")?
- **Scope-creep check:** does "explicitly out of scope" actually cover the things this feature will be tempted to also fix?
- **Contract sanity check:** does the API Contract's payload shape match existing conventions in this service? Are the exceptions exhaustive, or does the spec only cover the happy path plus one error?

- **Constitution compliance:** does the plan violate, or stay silent on, anything in constitution.md?
- **Overlap check:** does an existing spec already cover part of this? (Cheap to catch here; expensive to discover as a merge conflict of intent later.)
- **Dependency check:** are the specs this one "Builds on" or "Related to" actually in Approved/Released state, or is this spec quietly depending on something not yet real?

12.3 Turnaround and outcomes

- **Target SLA:** same working day for a spec with 5 or fewer acceptance criteria; 48 hours as the outer bound before it's an escalation, not a delay.
- **Outcomes are binary at the status level, even if feedback is nuanced:** *Approved* (status moves to Approved, plan drafting can start) or *Changes Requested* (status stays In Peer Review, spec is revised — bump a minor version marker in the Status line, e.g. **Draft v1.1**, so the revision history is visible in the file itself, not just in git blame).

12.4 Worked example

Reviewer reading [2fa-otp-login.spec.md](#) v1.0 catches that [2fa-otp-login.AC3](#) ("after 5 failed attempts... the otp_session_id is invalidated") doesn't say what happens if the user then logs in again correctly on the *first* try after invalidation — does the new session's counter start fresh, or does it inherit the old lockout? The spec is silent, which means the plan would have had to guess, and the agent generating from the plan would have guessed differently depending on the day. Comment goes back to the author; AC3 is revised to state explicitly that a new [otp_session_id](#) always starts its own counter at zero. Spec moves to v1.1, re-submitted, approved same day. **This is the gate doing its job** — it cost minutes, not the alternative, which is a QA-found defect three sprints later with a much less obvious root cause.



13. Architecture Standards (Plan Review, Gate 1 Continued)

- **Every plan is checked against `constitution.md` before task generation** — a plan that violates a non-negotiable is a Gate 1 rejection, not a Gate 2 comment.
- **Plans name the integration points and data model explicitly** — don't leave this for the agent to infer at implementation time.
- **Significant architectural decisions get an ADR**, not just a line in the plan (#22). A decision is "significant" if reversing it later costs more than a day of rework.
- **Plans state what they explicitly defer** — verify at Gate 2 that implementation didn't scope-creep into what the plan deferred.
- **No plan skips straight from spec to code**, except the trivial-change tier (config bumps, formatting, no behaviour change).

Plan template is in #10, Step 2, populated with the running example.

14. Repository Standards

- **One feature branch per spec** — PR scope stays reviewable against a bounded, known intent.
- **Branch naming:** `feature/<feature-slug>`, `fix/<issue-slug>`, `hotfix/<incident-slug>` — always traceable back to the spec file of the same slug (#9).
- **Squash-merge to main** — consolidates rapid agent-generated incremental commits into one human-paced logical unit.
- **No agent-signature suffixes in commit messages** ("Generated by Copilot," etc.) — configure IDEs/agents to disable this. A commit message referencing a task ID (`Implements 2fa-otp-login.T03`) is traceability, not attribution, and is encouraged.

- **PR size discipline:** if a PR can't be reviewed against its spec's acceptance criteria in one sitting, the spec was scoped too large — split it.
- **.ai-context/ and .agent/ live in the internal repo always.** Whether they also ship to a client-facing repo is an engagement-level decision — default to excluding them unless the SOW says otherwise.

15. AI Workspace — .agent/ and .ai-context/ Explained

None

```
project-root/
├── .agent/
│   ├── rules/
│   │   ├── int-standards.<stack>.md
│   │   ├── .agentignore
│   │   └── auto-log.md
│   └── workflows/
│       ├── code-review.md
│       ├── generate-tests.md
│       └── generate-plan.md
├── .ai-context/
│   ├── constitution.md
│   ├── project_context.md
│   ├── architecture.md
│   ├── BRD.md
│   ├── status.md
│   ├── specs/<feature-slug>.spec.md
│   ├── plans/<feature-slug>.plan.md
│   ├── tasks/<feature-slug>.tasks.md
│   ├── test_cases/
│   │   └── <feature-slug>.test_cases.md
```

```

| | └─ _integration.md
| └─ prompt_history.md
| └─ decisions/ADR-NNNN-<slug>.md
└─ src/
└─ tests/
└─ docs/
└─ README.md

```

.agent/ — the control plane. Governs *how the agent behaves*, independent of any one feature. Nothing project-specific-but-changing-often belongs here.

- `rules/int-standards.<stack>.md` — the always-on coding, error-handling, security, performance, and guardrail rules for that stack; the agent reads this on every session regardless of task.
- `rules/.agentignore` — keeps build artefacts, secrets, and heavy binaries out of the agent's context window; a token-cost and hallucination-surface control, not housekeeping.
- `rules/auto-log.md` — the instruction that makes the agent append to `prompt_history.md` after every completed task, so the audit trail exists without relying on the engineer remembering to write it.
- `workflows/*.md` — reusable prompt templates (`/code-review`, `/generate-tests`, `/generate-plan`) so the team isn't retyping the same instructions with slightly different wording every time, which is both wasted tokens and inconsistent output.

.ai-context/ — the knowledge base. Governs *what the agent (and the next engineer) needs to know* about this specific project.

- `constitution.md` — the non-negotiables (#8).
- `project_context.md` — one-page orientation: objective, architecture summary, stakeholders. What you'd hand a new hire on day one before anything else.
- `architecture.md` — the living system design: schemas, integration points, decisions in force. Must stay current or it actively misleads (#18).



- **BRD.md** — where a requirement is first written down, before it becomes a spec (#7).
- **status.md** — the state and progress board (#21) — new in this version, and arguably the artefact that makes SDD legible as *delivery management*, not just an engineering convention.
- **specs/, plans/, tasks/** — the per-feature artefact chain (#10), one file per feature, named by slug.
- **test_cases/** — split per-feature (#19) plus one integration/system file.
- **prompt_history.md** — the session-level agent audit trail, distinct from **status.md**'s human-curated daily summary.
- **decisions/** — ADRs, the "why," referenced from **architecture.md** and, when a decision changes a non-negotiable, from **constitution.md**.

16. Prompt Engineering

The core discipline: prompt by identity, not by description. Every artefact now carries a stable ID (#9) — a spec, an AC, an API endpoint, a task. A prompt that references the ID survives the spec being edited; a prompt that re-describes the feature from memory drifts the moment the spec does.

- **Bad:** "Implement the OTP verification endpoint, make sure it handles rate limiting."
- **Good:** "Implement **2fa-otp-login.T03** — must satisfy **2fa-otp-login.AC3** and match the exception table in **2fa-otp-login.API02**. Do not touch T01/T02, already merged."

Further rules:

- **One task, one prompt.** Don't ask for "the whole payment integration" — decompose per the tasks.md sequencing and review between steps.
- **Use .agent/workflows/ templates instead of freehand instructions** — token-efficient and produces consistent output.



- **State the acceptance criteria ID the agent should self-check against**, not just the action.
 - **Match model to task.** Boilerplate, regex, formatting, simple lint fixes → lighter/faster model. Architecture, complex logic, deep debugging → the heavier model.
 - **Don't iterate-and-hope.** If the first generation is significantly wrong, stop and fix the spec or plan rather than re-prompting repeatedly against the same ambiguity.
-

17. Context Engineering

- **Scope context to the task.** Tag only the specific files/folders needed, never let the agent scan the whole workspace for a localised change.
 - **Maintain `.agentignore` aggressively.**
 - **Flush chat history at feature/bug boundaries.** Start a new session per logical unit of work, not per day.
 - **Keep `architecture.md` current or don't trust it.** A stale doc actively misleads the next agent session.
 - **Context decay is the thing SDD exists to prevent** — if an agent needs to be re-told something twice, that something belongs in `.ai-context/`.
-

18. Development Standards

- Implementation only proceeds against an **approved tasks.md**, task by task, referenced by ID.
- **Small, verifiable increments** — generate, review, commit; don't let the agent run several tasks unattended and review the aggregate diff.
- **Stack-specific conventions live in `int-standards.<stack>.md`** — this document does not duplicate them.



- **Boundary and failure-path correctness is the engineer's job.** Agents nail the happy path; error handling, null/undefined edge cases, and partial-failure states need explicit acceptance criteria or they get skipped.
 - **Consistency with existing codebase patterns** — a human-eye check `.ai-context/architecture.md` alone doesn't guarantee.
 - **Performance characteristics that don't show up in a unit test** (N+1 queries, missing indexes, synchronous calls in a hot path) — agents optimise for "passes the test," not "performs at production load."
-

19. QA & Validation

19.1 One `test_cases.md`, or split it?

Decision: split it. A single master `test_cases.md` was workable for a handful of features; past roughly a dozen active specs, it becomes an unreviewable monolith, and it obscures the 1:1 traceability from a spec's AC to its own test cases, which is exactly the traceability #11.3 relies on. Structure:

- `.ai-context/test_cases/<feature-slug>.test_cases.md` — the spec-derived cases, owned and updated alongside that spec. This is largely a QA-expanded version of the spec's own Unit Test Cases table (#11.4) — same IDs, more scenario coverage (data variations, negative paths the AC didn't spell out).
- `.ai-context/test_cases/_integration.md` — the one master file that remains, reserved for cross-feature and system-level scenarios that don't map cleanly to a single spec (e.g., "policy purchase end-to-end: quote → payment → OTP-gated confirmation → regulatory report emitted"). This is where regression suites and full user-journey scenarios live.

This keeps Gate 1 and Gate 2 reviewable against one feature's scope while still giving QA a home for the scenarios that were always going to span features.

19.2 Standing rules



- **Test cases derive from acceptance criteria, not from reading finished code.**
- **Generated tests are a starting point QA validates, not a substitute for QA judgement** — QA owns data volumes, device/browser matrices, accessibility, and grey-area calls the spec didn't resolve.
- **UAT traces line-by-line to acceptance criteria (by AC ID).** A UAT failure with no matching AC means the spec was incomplete or Gate 1 missed a gap — either way, the spec gets updated, not just the code.
- **UAT-defect-to-spec-gap is a leading indicator**, tracked in retros, of whether Gate 1 is catching what it should.

20. Security Standards

- **Secrets and PII never enter a spec, plan, task, prompt, or prompt_history.md** — no exceptions.
- **Check generated code against the constitution, not just the spec.**
- **Vet AI-suggested dependencies before accepting them.**
- **SAST/DAST and dependency scanning run in the pipeline regardless of who (or what) wrote the code.**
- **Injection, auth boundaries, secret handling, least-privilege IAM, data-at-rest/in-transit** — Gate 2 checklist items with the same weight as functional correctness.
- **Treat any external content fed to an agent as untrusted input** — don't let instructions embedded in fetched content get executed as if the engineer typed them.

Worked example — 2fa-otp-login at Gate 2

Two things a reviewer actually caught on this feature, both directly traceable to constitution.md's Security Posture section (#8):

1. **PII-in-logs violation.** The agent's first pass at 2fa-otp-login.T02 generated:

js



JavaScript

```
logger.info(`OTP sent to ${phone}: ${otp}`);
```

This logs both the phone number and the OTP code itself, at info level — a direct violation of "no PII in logs at any level." Gate 2 required the fix:

js

JavaScript

```
logger.info('OTP dispatched', { otp_session_id, phone: maskPhone(phone) });  
// OTP code itself is never logged, at any level, full stop.
```

2. **Dependency vetting.** The agent suggested `npm install otp-generator-plus` for generating the 6-digit code. Rejected at review — unvetted, low-download third-party package doing something Node's built-in `crypto.randomInt(100000, 999999)` already does correctly and without a supply-chain dependency. [int-standards.node.md](#)'s Guardrails section now cites this as the standing example of why crypto-adjacent suggestions get extra scrutiny.
3. **Rate-limit key design.** The plan (#10) keys the Redis rate-limit counter by `otp_session_id`, not by raw phone number — deliberately, so the security-sensitive counter itself doesn't become a second place phone numbers are stored in plaintext.

21. State & Progress Management

SDD makes the artefact chain the source of truth for *what's being built*; it needs an equally deliberate mechanism for *what's happening right now* — otherwise "which spec is in development" is answered by asking around, which is exactly the kind of undocumented, non-repo-resident knowledge this whole standard exists to eliminate.





21.1 Spec/task lifecycle states

Every spec's **Status** field (#11.4) moves through a fixed set of states — nothing skips a state, including under hotfix pressure (#24 covers the compressed variant):

None

Draft → In Peer Review (Gate 1) → Changes Requested ↻ → Approved → Plan Drafted → Plan Reviewed → Tasks Generated → In Development → In QA → Ready for Release → Released (vX.Y.Z) → [Deprecated / Superseded]

Tasks carry a simpler state each (**Not Started / In Progress / In Review / Merged**), tracked as the checkbox state in the tasks.md file itself (#10, Step 3) — no separate task-tracking tool required, though **status.md** below rolls them up.

21.2 .ai-context/status.md — the board

A single file, updated by whoever last touched a spec, giving a same-day answer to "what's in flight" without a stand-up:

markdown

None

Project Status Board
Last updated: 2026-07-16

Active Specs
| Spec ID | Title | Status | Owner | Last Updated | Notes |
|---|---|---|---|---|---|
| 2fa-otp-login | OTP 2FA Login | In Development (T03/T04) | R. Sen | 2026-07-16 | T01-T02 merged; T03 blocked on Redis TTL config decision |
| payment-retry | Payment Retry Logic | In Peer Review | A. Roy | 2026-07-15 | Re-submitted v1.1 after idempotency-key feedback |

Daily Execution Log



2026-07-16

- ****2fa-otp-login****: T01-T02 merged, Gate 2 passed. T03 (rate-limit lockout) in progress — Redis TTL question raised with Architecture, answer expected tomorrow.
- ****payment-retry****: Spec revised to v1.1 per Gate 1 feedback (idempotency key now explicit in AC2). Re-submitted for review.

2026-07-15

- ****payment-retry****: Gate 1 review — changes requested, idempotency key handling under retry was ambiguous in AC2.

What this is not: a replacement for a proper delivery-tracking tool (Jira, Azure Boards, etc.) on engagements that already have one. Where such a tool exists, [status.md](#) should mirror the spec-level state, not compete with the ticket system for sprint-level task tracking — the value here is specifically that the state lives in the repo, next to the artefacts it describes, and survives a tool migration the way a Jira board doesn't.

22. Documentation Standards

- **architecture.md** — living system-design doc, updated whenever a plan introduces a new integration, data store, or significant decision.
- **decisions/ADR-NNNN-*.md** — the *why* behind significant technical choices, project-global IDs (#9).
- **Specs collectively are the feature-level changelog.**
- **prompt_history.md** — session-level audit trail, distinct from [status.md](#)'s human-curated daily summary and from the spec/plan/ADR decision layer.
- **README.md stays operational only** — setup, run, deploy.
- **Documentation currency is a standing Gate 2 checklist item.**

23. Release Management

Release management is where the spec-state machine (#21) and the codebase converge — a release is not just a git tag, it's a set of specs whose state changes together.

1. **A release is cut once its constituent specs are all **Ready for Release** in **status.md**** — this is the release-readiness source of truth, alongside your tag/branch convention.
2. **On cut, each included spec's status flips to **Released (vX.Y.Z)**** — permanent, dated, and traceable from the release tag back to the exact spec that produced it.
3. **Release notes are drafted from spec intents, not commit messages.**

Worked example:

markdown

None

Release v2.4.0 — 2026-07-20

Specs included

- 2fa-otp-login → Released (v2.4.0)
- payment-retry → Released (v2.4.0)

Release notes

- Policy holders can now enable OTP-based two-factor login on the portal (2fa-otp-login).
- Failed payment retries now use exponential backoff with idempotency keys, eliminating duplicate charges under retry (payment-retry).

status.md's Active Specs table drops both rows (or moves them to a **Released** archive table) the same day, keeping the board a true reflection of what's still in flight.



24. Hotfix Process

Rule: hotfixes get a compressed chain, never a skipped one — and they get their own identifier, linked back to the spec they patch.

1. **Incident gets a hotfix ID:** `HOTFIX-incident-slug`, e.g. `HOTFIX-2026-0714-otp-lockout-bug`.
2. **Write the lightweight spec first, even under pressure —** `.ai-context/specs/hotfix-2026-0714-otp-lockout.spec.md`, one paragraph: what's broken, correct behaviour, 1–2 AC. Status starts at `Emergency-Merged` (a hotfix-only state), and the spec's `Related` field points back to `2fa-otp-login`.
3. **Root-cause before patching.**
4. **Gate 1's reviewer-not-author rule may be skipped only in a genuine production emergency — Gate 2 is never skipped**, review within 24 hours regardless.
5. **Same working day, status moves `Emergency-Merged` → `Retro-Documented`** once the full spec entry is formalised.

Worked example, threaded from the running feature:

- **Incident:** Policy holders who fail OTP verification exactly 5 times, then immediately retry login and receive a *new* `otp_session_id`, are not being rate-limited — the lockout counter in `2fa-otp-login.T03` was keyed to the old session id only, so a new session resets it.
- **Root cause:** `2fa-otp-login.plan.md`'s data model keyed the Redis counter per-session rather than per-user, which the original spec's AC3 didn't disambiguate (see the Gate 1 example in #12.4 — this is the exact ambiguity that review caught for the *first* version of AC3, but a second, related ambiguity slipped through in the same area).
- **Hotfix:** Key the counter by a hash of the phone number instead of the session id — still no raw phone number stored, respecting `constitution.md`'s PII rule — so lockout persists across session churn.

- **Retro-spec:** Filed the same day as [hotfix-2026-0714-otp-lockout.spec.md](#), **Related:** [2fa-otp-login](#). Because this is the second AC3-adjacent gap, it's also flagged as an ADR candidate (#25) rather than treated as a one-off.
- **Release:** Shipped as a patch, included in the next release notes as "Fixed: OTP lockout not persisting across session retries ([hotfix-2026-0714-otp-lockout](#))."

25. Production Support

Incidents map to artefacts, not to ad hoc Slack threads — the mapping is what makes production learning durable instead of tribal:

| Trigger | Artefact created | Linked to |
|---|--|---|
| Incident traces to a spec gap | Hotfix spec (#24) or standard bug-fix spec | Original spec via Related ; logged in status.md Daily Execution Log |
| Incident traces to an implementation defect, spec was fine | Lightweight bug-fix spec, no spec-gap flag | Original spec unchanged |
| Same class of incident recurs (3rd occurrence in the same area) | ADR trigger | New ADR-NNNN , referenced from architecture.md ; constitution.md amended if it changes a non-negotiable |
| Genuine new requirement surfaces via support | New BRD entry | BRD.md , feeds a new spec through the normal lifecycle (#7–#11) |





Worked example: OTP SMS dispatch via Twilio times out under load three times in one month. Third occurrence trips the ADR trigger:

markdown

None

ADR-0007: Async retry queue for OTP dispatch

Context

Direct/synchronous Twilio calls in the OTP dispatch path have caused three P2 incidents in one month under load spikes, each traced to Twilio-side latency, not our code.

Decision

Move OTP dispatch onto a dedicated retry queue (existing Kafka topic, consumer with exponential backoff, 3 retries) rather than a direct call in the request path.

Consequences

OTP delivery latency increases slightly on the retry path (acceptable per a Product-agreed 30s SLA); dispatch failures no longer surface as request-path errors. Does not solve: Twilio-side outages longer than the retry window, which still require the existing incident process.

architecture.md's Integrations section is updated to reference ADR-0007, and constitution.md's Non-Functional Baselines gains a new line: "OTP dispatch must never block the login request thread" — a rule the next spec touching this area inherits automatically, instead of every future engineer having to rediscover it the hard way.

26. Engineering KPIs

Track from rollout; do not invent targets before a 90-day baseline exists.

| KPI | What it tells you |
|-----|-------------------|
|-----|-------------------|



| | |
|------------------------------------|---|
| Spec-first compliance rate | % of merged PRs with an approved spec at merge time |
| Rework rate | % of merged features needing a 14-day follow-up fix traceable to a spec gap vs. a genuine new requirement |
| Spec-review cycle time | Median time from spec submission to Gate 1 approval |
| UAT-defect-to-spec-gap ratio | Leading indicator of whether Gate 1 is catching ambiguity before QA |
| Hotfix retroactive-spec compliance | % of hotfixes with a same-day formalised spec entry |
| PR cycle time (open → merge) | Whether two-gate review is adding friction disproportionate to its value |
| Architecture doc currency | % of merged plans that touched architecture.md/ADRs when they should have |
| Repeat-incident-to-ADR rate | % of 3rd-occurrence incidents that actually produced an ADR, vs. quietly patched again |

Own these in Technology Office quarterly review.

27. Definition of Ready / Definition of Done

Definition of Ready (spec → development)

- Intent fits in one unambiguous paragraph
- Acceptance criteria are testable and individually IDed (`<slug>.AC#`)
- API Contract section present and complete, if the feature has an API surface



- Context artefacts linked (architecture, related specs, BRD entry)
- Explicitly out-of-scope items named
- Non-functional constraints named, sourced from constitution.md
- Gate 1 reviewer assigned, not the author
- Status is **Approved**, not **Draft** or **Changes Requested**

Definition of Done (implementation → merge)

- All acceptance criteria verified individually by ID, not "looks reasonable"
- Tests written first, confirmed Red, then Green
- No AI-attribution in comments or commit messages (task-ID references are fine)
- No secrets, PII, or client-confidential data in spec/plan/tasks/code
- Security checklist passed against constitution.md (#20)
- architecture.md / ADR updated if the change warrants it
- Gate 2 code review complete, categorised feedback addressed
- **test_cases/<slug>.test_cases.md** and spec Status updated
- **status.md** updated same day (task moved to Merged, notes added)

28. Anti-Patterns

Vibe coding — the primary anti-pattern this document exists to eliminate:

Prompting an agent directly against a codebase with no spec, no plan, no persistent context — "build me a login page," iterate, patch, re-prompt — on any branch that gets reviewed, merged, or demoed. Exploratory local spikes are the only exception, and they graduate to a spec the moment they show promise.

| Anti-pattern | Why it's prohibited |
|--|--|
| Mega-prompting ("build the whole feature") | Produces a large diff with dozens of unreviewed implicit decisions |

| | |
|--|--|
| Re-prompting repeatedly against an ambiguous spec | Vibe coding with extra steps — fix the spec, don't iterate around it |
| Retrofitting tests after implementation | Inverts the Red-phase check that catches wrong-behaviour-by-design |
| Dumping the whole repo into agent context | Token waste and increased hallucination surface |
| Secrets or real customer data in a spec/prompt | Creates a new, greppable category of exposure that didn't exist before |
| AI-attribution in comments or commits | Client-trust and IP-hygiene violation |
| Skipping Gate 1 because "the spec looks obviously right" | The gate is cheap specifically so it never needs skipping — and it's now the <i>most</i> consequential gate (#12), not the least |
| Letting the agent pick architecture unbounded | Plan review exists precisely to catch constitution violations before they're built |
| Treating a stale architecture.md as current | Actively misleads the next agent session — worse than no doc |
| Prompting by description instead of by ID | Drifts the moment the artefact is revised (#16) |
| Letting <code>status.md</code> go stale | Recreates the exact "ask around to find out what's in flight" problem this artefact exists to remove |
| "The AI wrote it" as a review defence | Never a valid answer — the engineer who ran the agent owns it |

29. Templates





constitution.md — full skeleton

markdown

None

Project Constitution

Testing Discipline

<frameworks, coverage floor, test-first scope>

Security Posture

<PII/logging rules, secret management, auth baseline>

Architectural Constraints

<approved datastores, approved integration patterns, no new services without ADR>

Non-Functional Baselines

<latency targets, availability targets, RPO/RTO>

Versioning Rules

<semver, breaking-change process, deprecation window>

spec.md — full skeleton

See #11.4 (includes API Contract and Unit Test Cases sections).

plan.md — full skeleton

markdown

None

Plan: <Feature Name>

Derived From

.ai-context/specs/<feature-slug>.spec.md



Architecture Approach

<components touched, new vs. existing, integration points>

Data Model

<schema changes, migrations, if any>

Constitution Check

- [] No new datastore introduced without ADR
- [] Testing discipline matches constitution.md
- [] Security posture matches constitution.md

Explicitly Deferred

- <item, with reason>

Sequencing

1. <high-level build order>

tasks.md — full skeleton

markdown

None

Tasks: <Feature Name>**## Derived From**

.ai-context/plans/<feature-slug>.plan.md

Sequence

- [] <slug>.T01 — <independently verifiable unit of work> — Acceptance: <AC ID(s)>
- [] <slug>.T02 — ...

ADR — full skeleton



markdown

None

ADR-NNNN: <Decision Title>

Context

<Forces and constraints driving the decision>

Decision

<What was decided>

Consequences

<Positive and negative, including what this doesn't solve>

status.md — full skeleton

markdown

None

Project Status Board

Last updated: <date>

Active Specs

| Spec ID | Title | Status | Owner | Last Updated | Notes |

|---|---|---|---|---|---|

Daily Execution Log

<date>

- <spec-id>: <what moved today, blockers, decisions needed>

Hotfix spec — full skeleton

markdown

None

Spec: HOTFIX-<incident-slug>

Status

Emergency-Merged → Retro-Documented (same day)

Related



.ai-context/specs/<original-feature-slug>.spec.md

What Was Broken

<one paragraph>

Root Cause

<one paragraph, not just the symptom>

Correct Behaviour (Acceptance Criteria)

1. <failing scenario now succeeds>
2. <existing regression suite still passes>

Fix Summary

<what changed, why it respects constitution.md>

30. Checklists — Quick Reference

Gate 1 — Spec Peer Review

- ☐ Reviewer ≠ author
- ☐ Intent is one unambiguous paragraph
- ☐ Every AC is given/when/then and individually IDed
- ☐ API Contract complete (payload, success shape, exception table) if applicable
- ☐ Out-of-scope items explicit
- ☐ Plan (if attached) checked line-by-line against constitution.md
- ☐ Related/Builds-on specs are actually in Approved/Released state
- ☐ No overlap with an existing spec
- ☐ Security/Architecture sign-off obtained where the constitution requires it
- ☐ Status updated to Approved or Changes Requested (never left ambiguous)

Gate 2 — Code Review

- ☐ Each AC verified individually against the diff, by ID
- ☐ No AI-attribution in comments/commits (task-ID references are fine)
- ☐ Security checklist (below) passed



- ☐ architecture.md / ADR updated if warranted
- ☐ Tests were written first and confirmed Red before Green (not retrofitted)
- ☐ [status.md](#) and spec Status updated same day
- ☐ Standard PR review (readability, naming, DRY) otherwise applies

Security Checklist (used at Gate 2 and on every hotfix)

- ☐ No PII in logs at any level
- ☐ No secrets, credentials, or tokens hardcoded or logged
- ☐ All new/changed endpoints have an explicit rate-limit decision
- ☐ New dependencies vetted (maintained, real, approved) before entering the manifest
- ☐ Auth boundaries and least-privilege IAM checked, not assumed
- ☐ Data-at-rest and in-transit handling matches constitution.md
- ☐ SAST/DAST and dependency scan run and clean (or exceptions explicitly signed off)

Release Checklist

- ☐ All constituent specs are [Ready for Release](#) in status.md
- ☐ Release notes drafted from spec intents, not commit messages
- ☐ Each included spec's Status flipped to [Released \(vX.Y.Z\)](#)
- ☐ [status.md](#) Active Specs table updated (rows moved/archived)
- ☐ Client-repo sync (if applicable) run as a scripted CI/CD step

Hotfix Checklist

- ☐ Hotfix ID assigned ([HOTFIX-**<incident-slug>**](#))
- ☐ Lightweight spec written before the patch, root cause identified (not just symptom)
- ☐ [Related](#) field links back to the original spec
- ☐ Gate 2 not skipped, even post-hoc; reviewed within 24 hours
- ☐ Same-day retro-spec formalisation, status moved to Retro-Documented
- ☐ 3rd occurrence of the same incident class checked against the ADR trigger



Production Support / Incident Triage Checklist

- ☐ Classified: spec gap / implementation defect / genuine new requirement
- ☐ Correct artefact created per the mapping in #25 (hotfix spec / bug-fix spec / ADR / BRD entry)
- ☐ Original spec's [Related](#) or [architecture.md](#) updated to reflect the learning
- ☐ Occurrence count checked — 3rd time in the same area triggers an ADR, not another silent patch
- ☐ [status.md](#) Daily Execution Log entry added same day

INT.

